Защита капстоуна: системное администрирование

ФИО: Казачкин Даниил Михайлович

Репозиторий: github.com/dreykanbern/mephi-masters/tree/main/sysadmin/homework

Дорожка: Продвинутый

1. Git-репозиторий

Ответ GitHub API с private: False подтверждает публичность репозитория. История кода решения содержит больше трёх осмысленных коммитов и показывает развитие от скрипта до настройки сервиса.

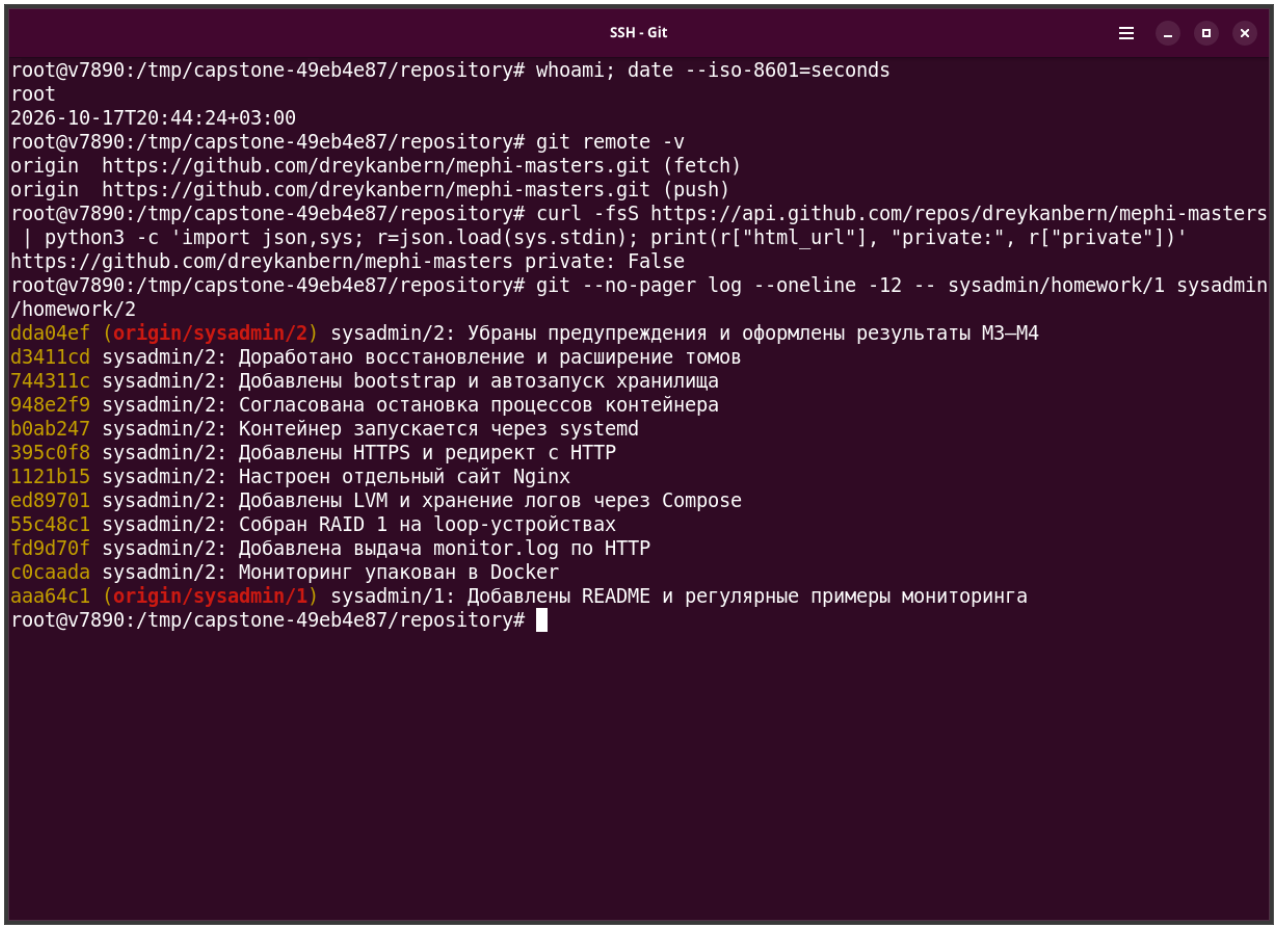


Рисунок 1. Публичность репозитория и история коммитов

2. Скрипт автоматизации

В SSH-сессии под root скрипт за шесть секунд записал два замера в monitor.log с интервалом в пять секунд, что подтверждает повторный сбор и сохранение показателей. Код 124 означает ожидаемую остановку бесконечного цикла командой timeout.

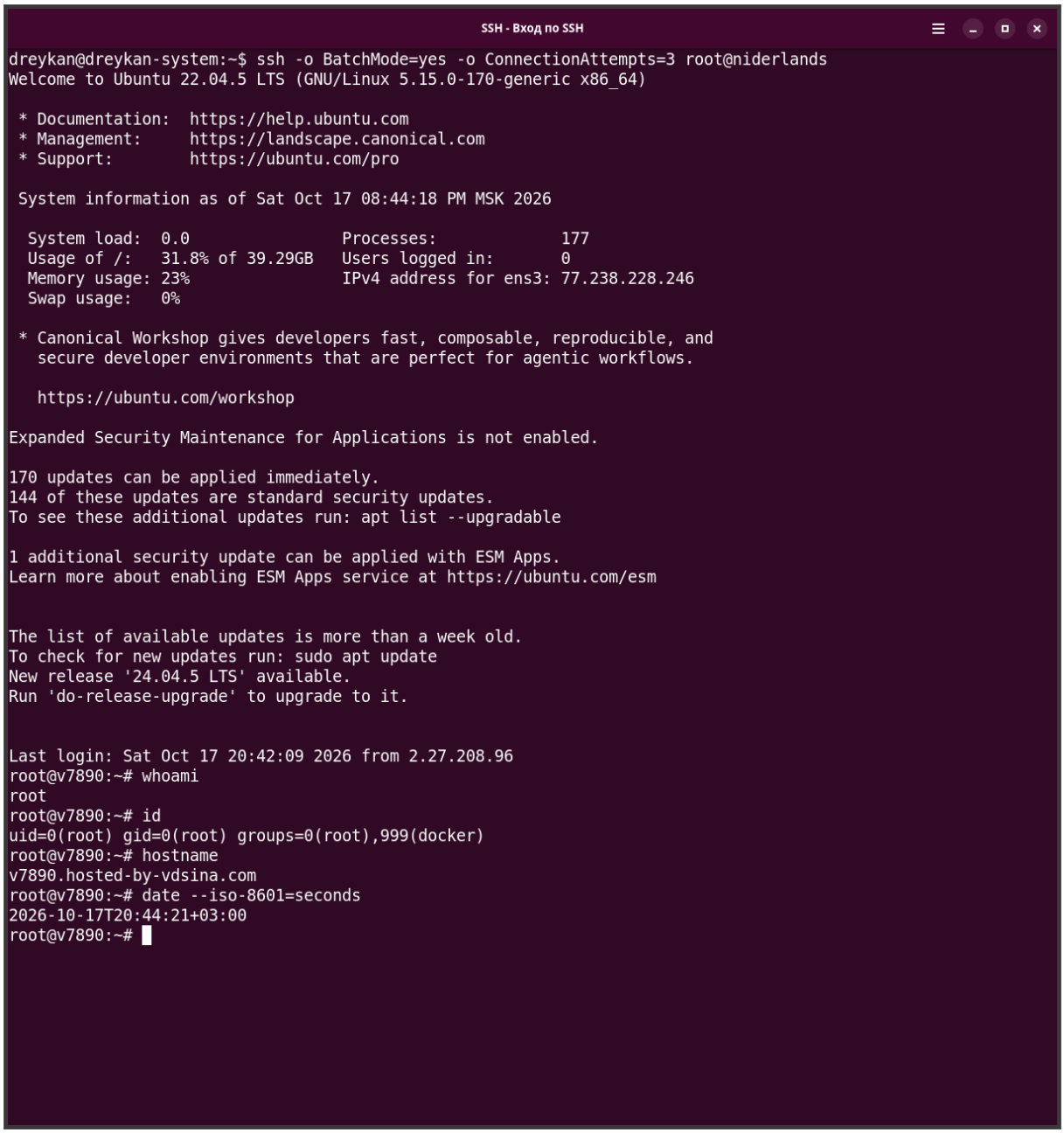


Рисунок 2. Подключение к серверу по SSH под root

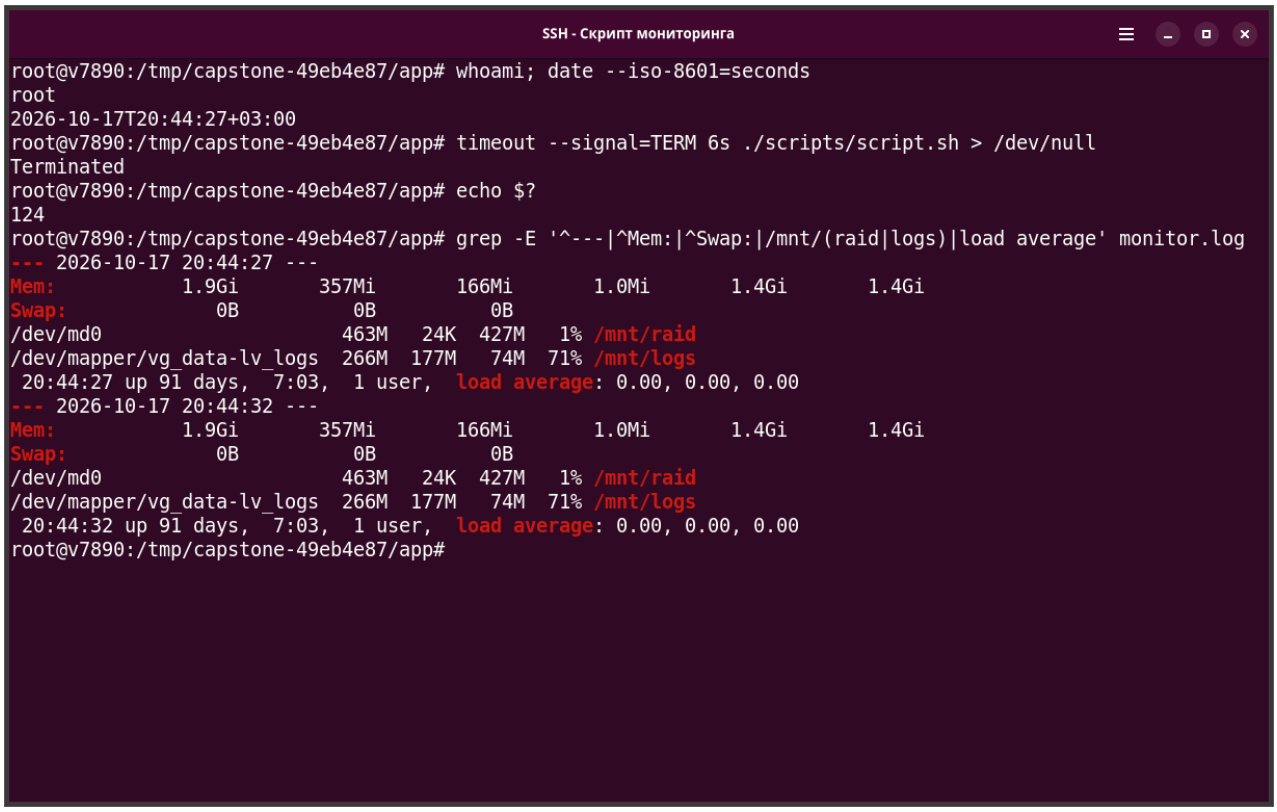


Рисунок 3. Запуск скрипта мониторинга

3. Dockerfile и контейнер

Успешное завершение docker build и состояние Up в docker ps подтверждают сборку образа и запуск контейнера из него. Ответ HTTP 200 показывает, что запущенный контейнер отдаёт monitor.log.

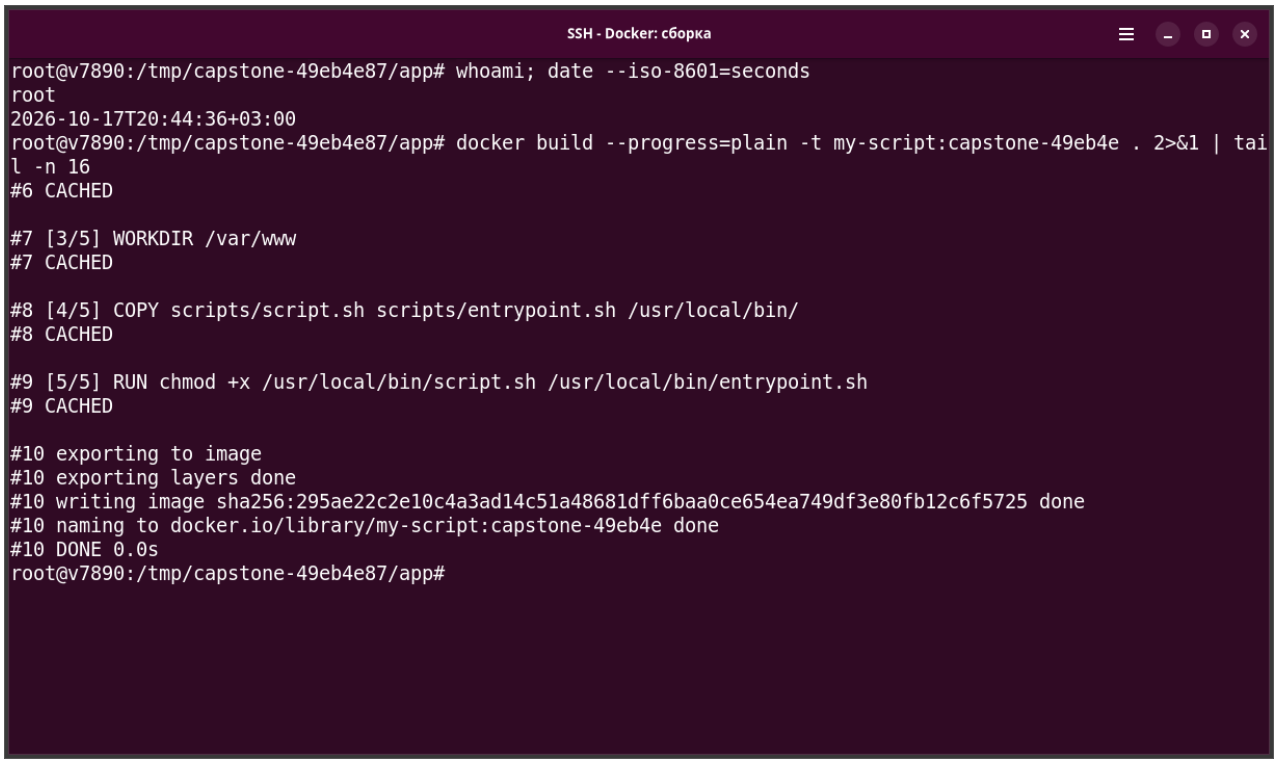


Рисунок 4. Сборка Docker-образа

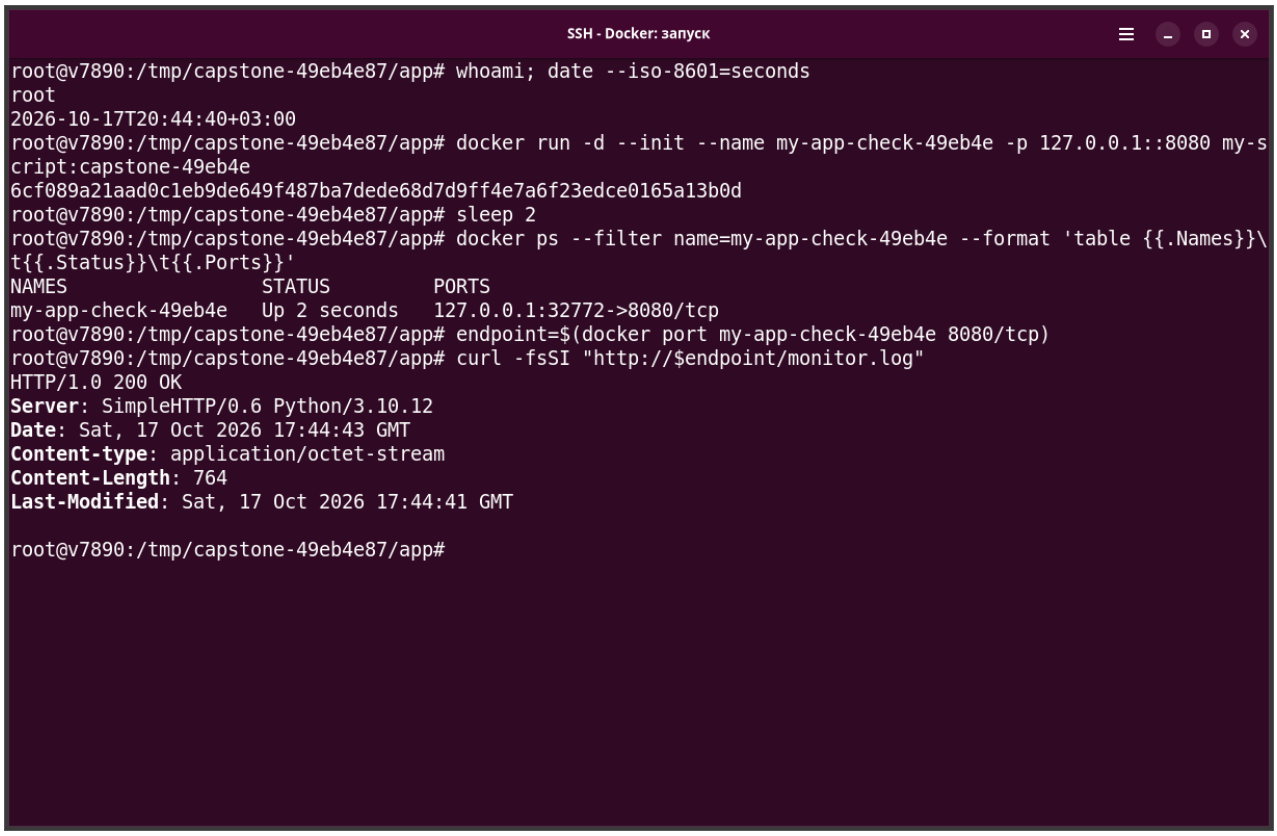


Рисунок 5. Запуск контейнера и ответ HTTP-сервера

4. RAID и LVM

Статус active raid1 и [UU] подтверждают работу RAID 1 с двумя участниками, а lvs показывает активный том lv_logs размером 300 MiB. Вывод df -h подтверждает монтирование RAID в /mnt/raid и LVM в /mnt/logs.

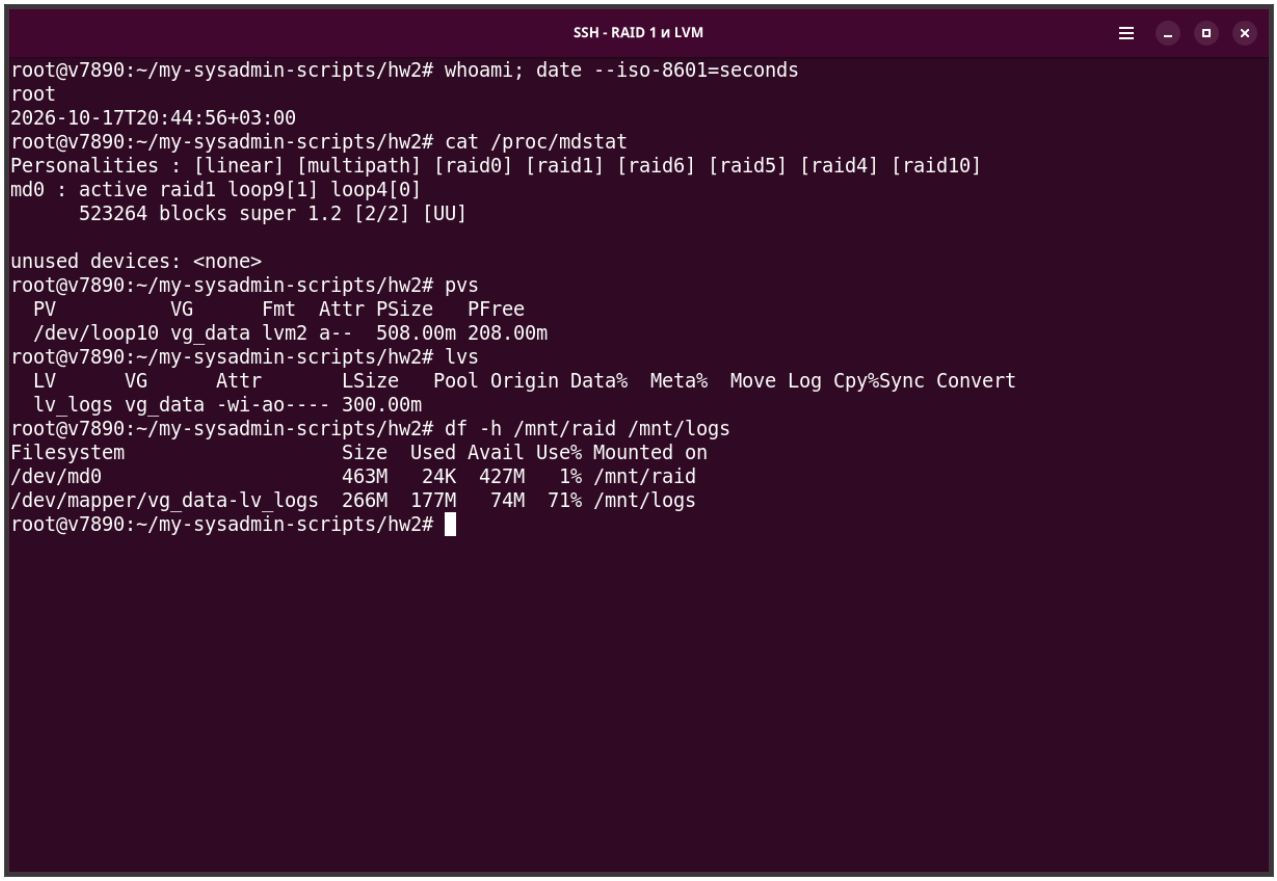


Рисунок 6. RAID 1, LVM и точки монтирования

5. Nginx

Сообщения syntax is ok и test is successful подтверждают успешную проверку nginx -t. Строка proxy_pass http://127.0.0.1:8080 показывает, что Nginx настроен как обратный прокси к приложению.

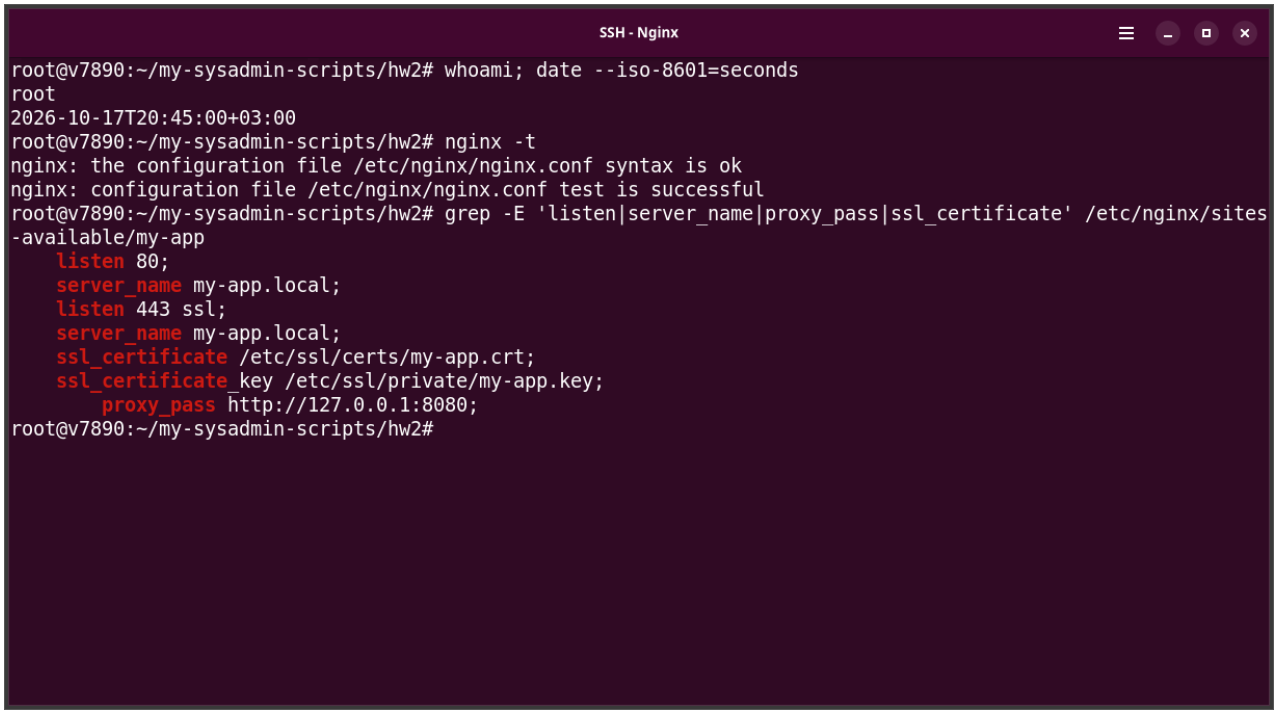


Рисунок 7. Проверка конфигурации Nginx

6. TLS и HTTPS

Ответ HTTP 200 на HTTPS-запрос подтверждает доступность monitor.log через TLS в Nginx. Для проверки самоподписанного сертификата использован curl с флагом -k; ниже также показаны имя и срок действия сертификата.

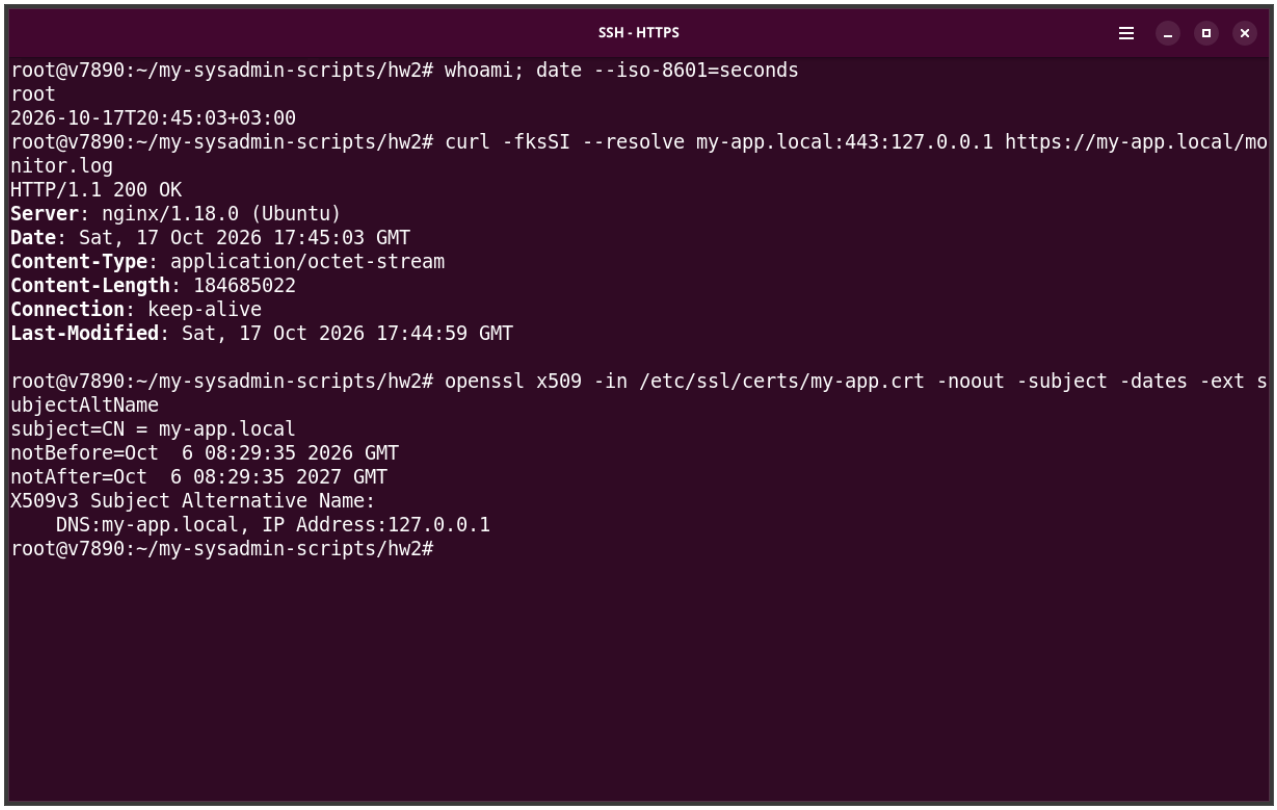


Рисунок 8. HTTPS и сертификат

7. Служба systemd

Статус active (running) подтверждает, что служба my-app работает. Вывод is-enabled со значением enabled подтверждает автозапуск приложения и службы подготовки хранилища.

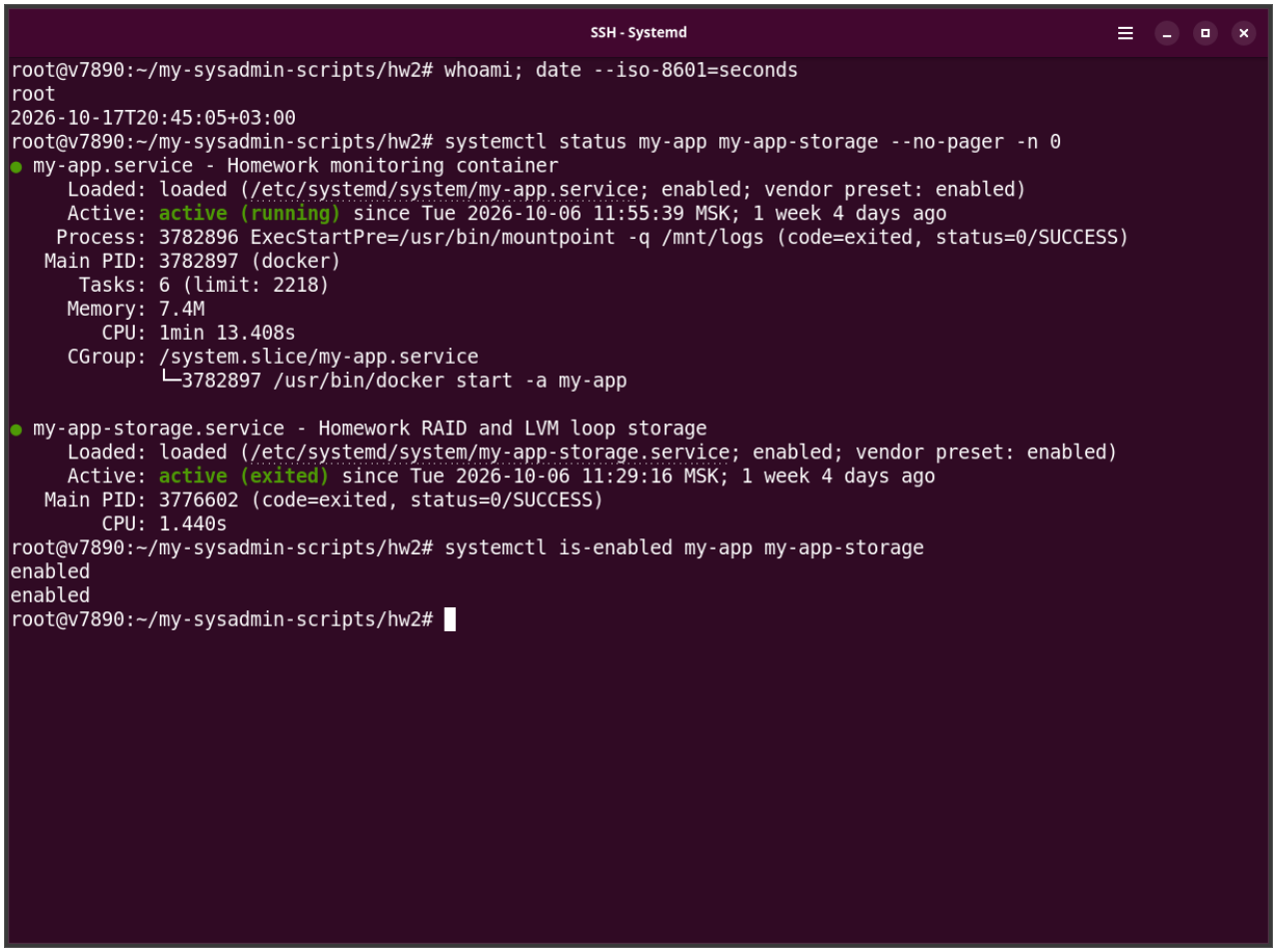


Рисунок 9. Состояние служб и автозапуск

8. Логи

Одинаковая метка запроса и код 200 в journalctl и access.log подтверждают прохождение запроса через Nginx до приложения. Последняя запись error.log сделана раньше тестового запроса: новых ошибок при этом обращении не записано.

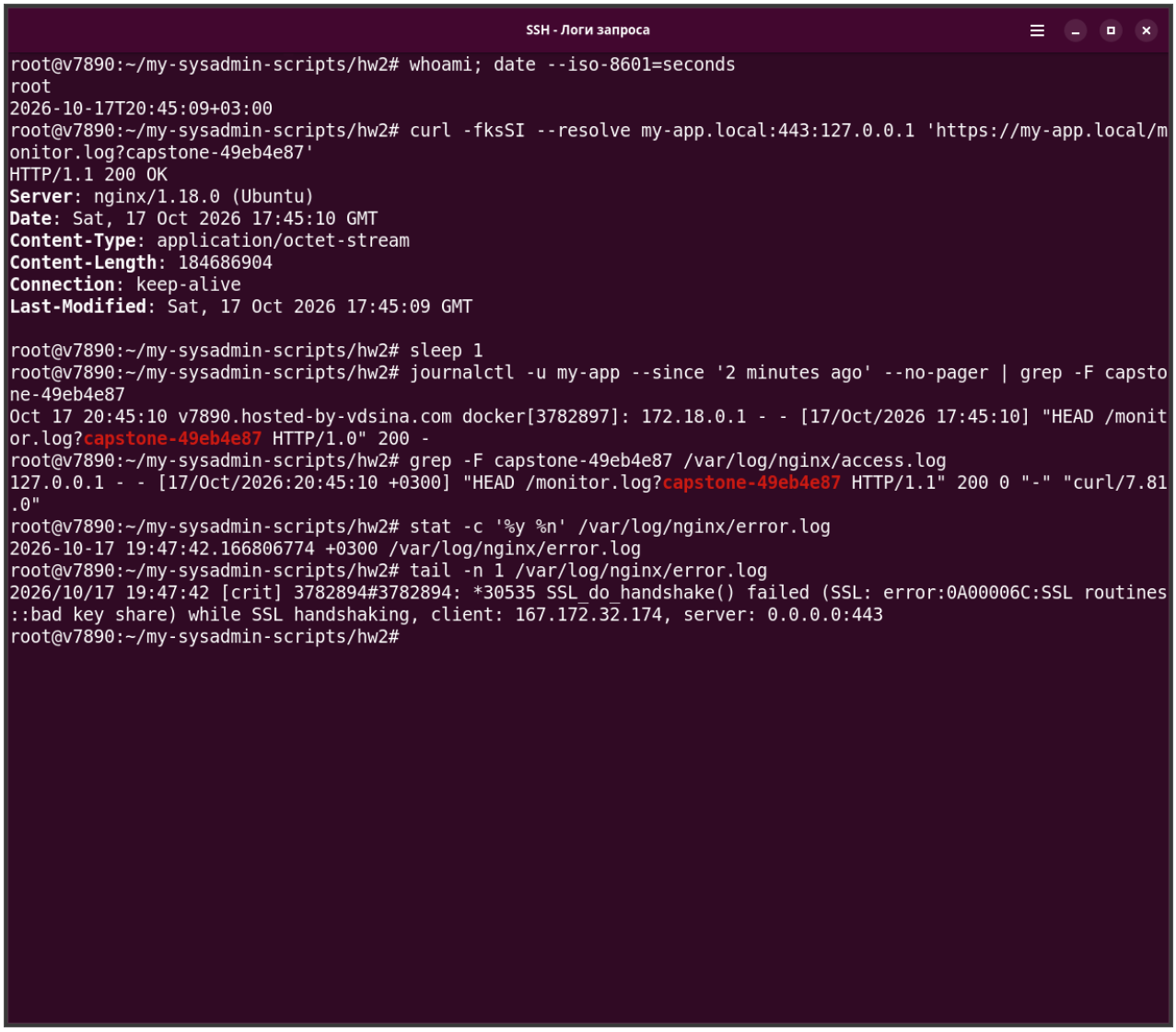


Рисунок 10. Тестовый запрос в журналах

Архитектурные решения

Выбор варианта скрипта

В работе M1–M2 «Паспорт системы» + «Скрипт в Git» выбрал вариант Б «Мониторинг ресурсов». На этом же VPS у меня работает VPN-сервис, поэтому мониторинг был полезен и для действующей машины: мне нужно видеть, сколько памяти и места доступно и как меняется нагрузка. Результат сразу проверяется по новым записям и меткам времени в журнале. Скрипт только считывает системные показатели и дописывает собственный журнал.

Этот вариант удобно продолжить в капстоуне: monitor.log стал общим результатом работы скрипта, файлом на LVM и ресурсом, который контейнер отдаёт через Nginx по HTTPS. Так все компоненты используются в одной задаче.

Выбор языка реализации

В работе мне привычны TS, Python и Go, поэтому скрипт мог быть написан на любом из этих языков. По работе у меня уже был опыт настройки окружения в GitLab и запуска скриптов с Docker и Docker Compose. Поэтому Bash был для меня предсказуемым выбором: я уже понимал, как запускать такой скрипт в контейнере, передавать ему параметры окружения и читать его вывод. Для этой задачи нужно раз в пять секунд запускать готовые free -h, df -h и uptime и дописывать их вывод в файл. Сложной обработки данных здесь нет.

RAID 1 и LVM

Собрал оба инструмента: RAID 1 из двух учебных файлов по 512 MiB, смонтированный в /mnt/raid, и отдельный LVM-том vg_data/lv_logs на третьем файле. Журнал monitor.log находится в /mnt/logs/my-app на LVM.

RAID 1 обеспечивает отказоустойчивость за счёт двух копий данных: при отказе одного участника данные остаются на другом. Цена дублирования - полезная ёмкость равна объёму одного из двух одинаковых дисков. В этой работе loop-устройства демонстрируют механику RAID на файлах одного VPS и не защищают от потери его физического диска.

LVM обеспечивает гибкое управление объёмом и само по себе данные не дублирует. В работе M3–M4 «Контейнер + том» + «Сервис за reverse proxy с TLS» я расширил lv_logs с 200 до 300 MiB вместе с файловой системой, сохранив файлы.

Получение сертификата

Для этого учебного стенда использовал локальное имя my-app.local и не настраивал отдельный публичный домен и ACME. Задача состояла в том, чтобы подключить TLS в Nginx и проверить доступ к приложению по HTTPS. Поэтому выбрал самоподписанный сертификат: его можно выпустить локально через OpenSSL при запуске bootstrap.sh и повторить развёртывание на другой машине без обращения к удостоверяющему центру.

Компромисс такого решения - клиент не доверяет сертификату автоматически. В проверке использовал curl -k, который отключает проверку сертификата. Для публичного сервиса выбрал бы сертификат доверенного центра с автоматическим обновлением, чтобы пользователям не приходилось добавлять доверие вручную.

Альтернативы

TS. Мог написать сборщик на привычном TypeScript, но тогда понадобилась бы среда выполнения TS/JavaScript в образе. Для периодического запуска трёх системных команд это добавило бы зависимость без новой функции.

Python. Он уже установлен в контейнере для HTTP-сервера, поэтому дополнительной среды не потребовалось бы. Python удобен для разбора показателей, записи JSON и расчётов. В текущем варианте нужно только сохранить готовый вывод команд, с чем Bash справляется напрямую.

Go. Мог объединить мониторинг и HTTP-сервер в одном бинарном файле. Но для этой работы затраты на отдельную программу не окупались: пришлось бы реализовать цикл сбора, запись журнала, обработку ошибок и HTTP-обработчик, проверить их и добавить сборку бинарного файла в Dockerfile. При текущей задаче из трёх команд раз в пять секунд и выдачи одного файла мне не требовался выигрыш в производительности. Поэтому использовал небольшой Bash-скрипт и готовый python3 -m http.server: такое решение было быстрее собрать и проще проверить в рамках задания.

Готовые утилиты. sar и atop использовал как референсы по периодическому сбору показателей и сохранению истории. sar подходит для анализа системной статистики, atop - для разбора потребления ресурсов отдельными процессами.

В этой работе нужен небольшой текстовый журнал памяти, свободного места на дисках и средней нагрузки, который можно читать через HTTPS. Эти данные уже дают free, df и uptime. Подключение sar или atop добавило бы пакеты в образ и потребовало бы выбрать режимы и формат вывода. Поэтому сохранил скрипт из работы M1–M2 «Паспорт системы» + «Скрипт в Git»: он явно показывает весь путь от запуска команд до записи в monitor.log. Компромисс - мой скрипт не разбирает нагрузку по отдельным процессам и собирает только выбранные показатели; для более глубокой диагностики использовал бы готовые инструменты.

Разовый запуск можно выполнить через docker run. Постоянную конфигурацию контейнера описал в Compose, а запуск после подготовки хранилища и перезапуск при сбое поручил systemd. Политику перезапуска Compose отключил, чтобы контейнером управлял один механизм.

Доработки

Сложность автозапуска была в том, чтобы восстановить хранилище до запуска контейнера после перезагрузки. Файлы учебных дисков сохраняются, но loop-устройства нужно подключать заново, причём их номера могут измениться. Если запустить приложение до сборки RAID, активации LVM и монтирования /mnt/logs, каталог с журналом ещё не будет готов.

Для этого добавил службу my-app-storage и зависимости Requires/After в my-app. Скрипт получает актуальные имена loop-устройств через losetup --show и ждёт завершения udev перед проверкой массива. Это позволяет избежать гонки с автоматической сборкой RAID. В my-app также проверяется точка монтирования до старта контейнера. Восстановление после перезагрузки проверил на отдельной VM в рамках работы M3–M4 «Контейнер + том» + «Сервис за reverse proxy с TLS».

В entrypoint доработал совместную остановку мониторинга и HTTP-сервера: при неожиданном завершении одного процесса контейнер завершается с ошибкой, после чего systemd может его перезапустить.